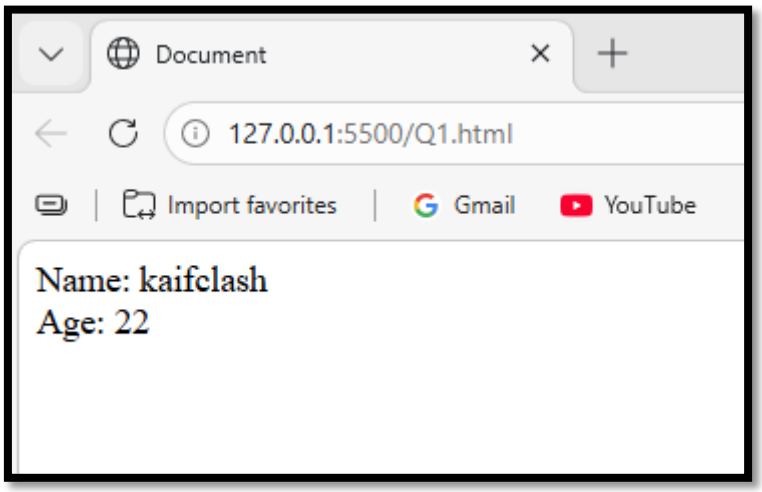
1	**Create class as Person with name and age as data members. Write parameterized constructor for set the value of data and getPerson method for show the data.**
Code	

```
<script>
        class Person{
            constructor(name,age){
                this.name=name;
                this.age=age;
            }
            getPerson(){
                document.write("Name: ",this.name);
                document.write("<br>Age: ",this.age);

            }

        }
        const p1=new Person("kaifclash",22);
        p1.getPerson();
    </script>
```

Output

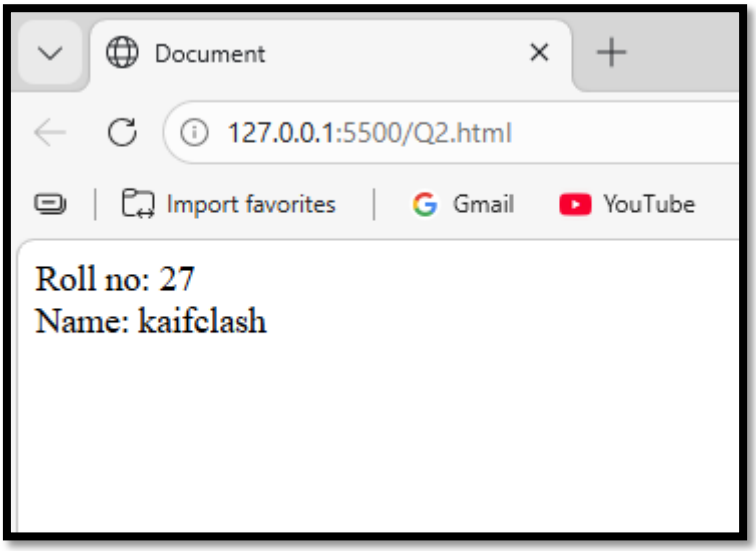


2	**Create class as Student with name and rno as data members. Write default constructor for initialize the value as blank. Set the value of data using setStudent method and show the value of data using getStudent method.**
Code	

```
<script>
        class Student{
            constructor(){
                this.rno="";
                this.name="";
            }
            setStudent(rno,name){
                this.rno=rno;
```

```
                this.name=name;
            }
            getStudent(){
                document.write("Roll no: ",this.rno);
                document.write("<br>Name: ",this.name);

            }
        }
        const s1=new Student();
        s1.setStudent(27,"kaifclash");
        s1.getStudent();
    </script>
```

Output



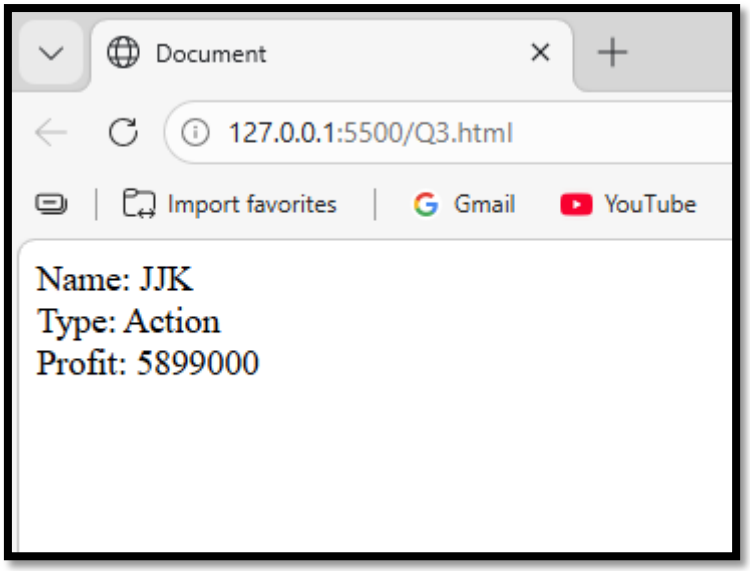
3 **Create class company with name, type and profit as data members. Write getCompany and setCompany method for set and show the data.**

Code

```
<script>
        class Company{
            setCompany(name,type,profit){
                this.name=name;
                this.type=type;
                this.profit=profit;

            }
            getCompany(){
                document.write("Name: ",this.name);
                document.write("<br>Type: ",this.type);
                document.write("<br>Profit: ",this.profit);
            }
        }
        const c=new Company();
        c.setCompany("JJK","Action",5899000);
        c.getCompany();
```

```
    </script>
```

Output



4

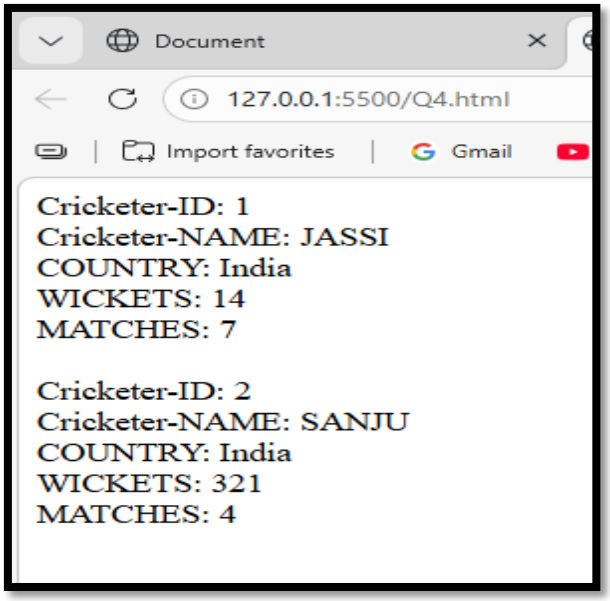
Create parent class as Cricketer with cid, cname and country as data member.
Write two methods as getCountry and setCountry.
Create child class as Bowler with wickets and matches as data member.
Write two methods as getBowler and setBowler.
Create another child class as Batsman with runs and matches as data members. Write two methods as getBatsman and setBatsman.
Create one object of Bowler class and one object of Batsman class and show all information.

Code

```
<script>
        class Cricketer{
            setCountry(cid,cname,country){
                this.cid=cid;
                this.cname=cname;
                this.country=country;
            }
            getCountry(){
                 document.write("Cricketer-ID: ",this.cid);
                 document.write("<br>Cricketer-NAME: ",this.cname);
                 document.write("<br>COUNTRY:
",this.country,"<br>");
            }
        }
        class Bowler extends Cricketer{

            setBowler(wickets,matches){
                this.wickets=wickets;
                this.matches=matches;
            }
```

```
            getBowler(){
                this.getCountry();
                document.write("WICKETS: ",this.wickets);
                document.write("<br>MATCHES: ",this.matches,"<br>");
            }

        }
         class Batmans extends Cricketer{

            setBatmans(run,matches){
                this.run=run;
                this.matches=matches;
            }
            getBatmans(){
                this.getCountry();
                document.write("WICKETS: ",this.run);
                document.write("<br>MATCHES: ",this.matches,"<br>");
            }

        }

        const b1=new Bowler();
        b1.setCountry(1,"kaif","India");
        b1.setBowler(20,7);
        b1.getBowler();

        const b2=new Batmans();
        b2.setCountry(2,"kaifclash","India");
        b2.setBatmans(20,7);
        b2.getBatmans();
    </script>
```

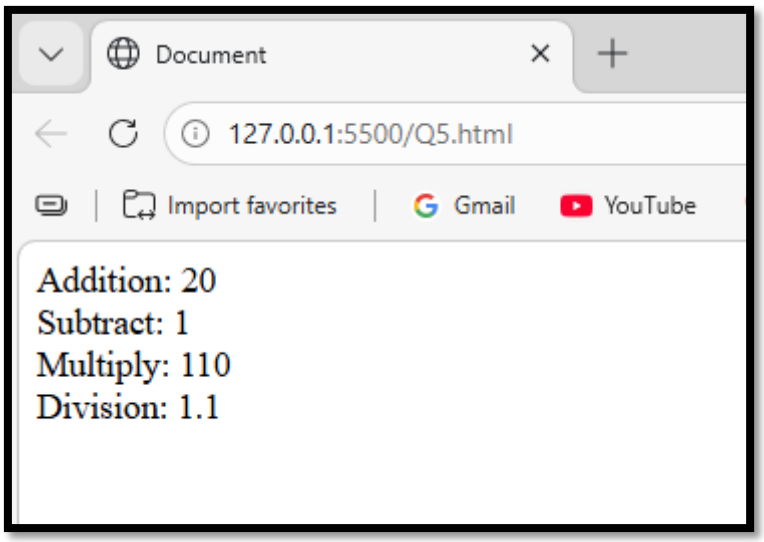
Output



5	**Write down arrow functions for different arithmetic operations.**
Code	

```
<script>
        const add=(a,b)=>a+b;

        const div=(a,b)=>a/b;

        const sub=(a,b)=>a-b;

        const multi=(a,b)=>a*b;

        var a=add(10,10);
        document.write("Addition: ",a);

        var b=sub(11,10);
        document.write("<br> Subtract: ",b);

        var c=multi(11,10);
        document.write("<br> Multiply: ",c);

        var d=div(11,10);
        document.write("<br> division: ",d);

    </script>
```

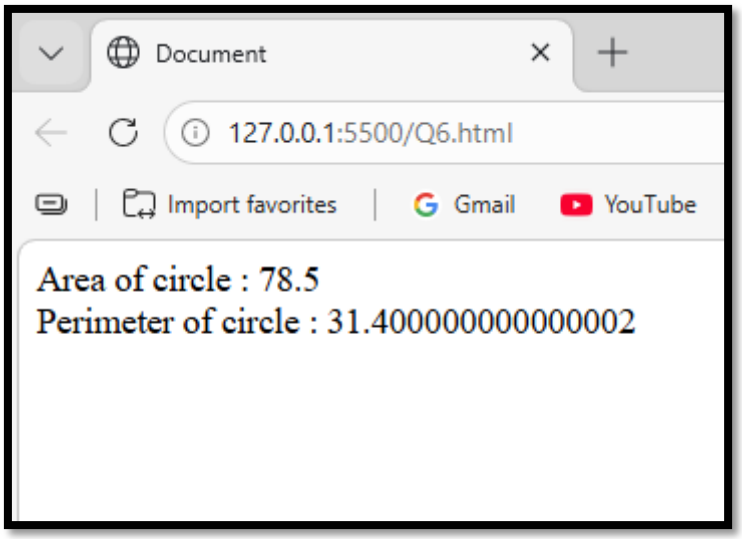
Output	



6	**Write down arrow functions for finding the area and perimeter of the circle.**
Code	

```
<script>
      const area=(r)=>3.14*r*r;
      var c=area(5);
      document.write("Area of circle : ",c,"<br>");

      const peri=(r)=>2*3.14*r;
       var d=peri(5);
      document.write("Perimeter of circle : ",d,"<br>");
  </script>
```

Output



7	**Write different arrow functions to solve following equations.**

a. $X^2 + Y^2 + XY$

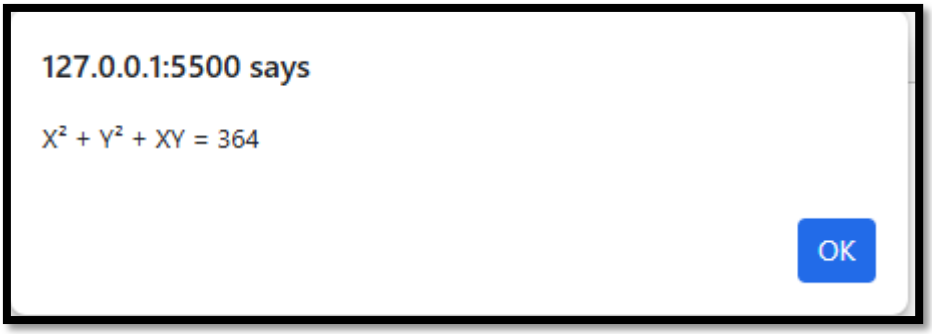
b. (X + Y) - (Y + Z)

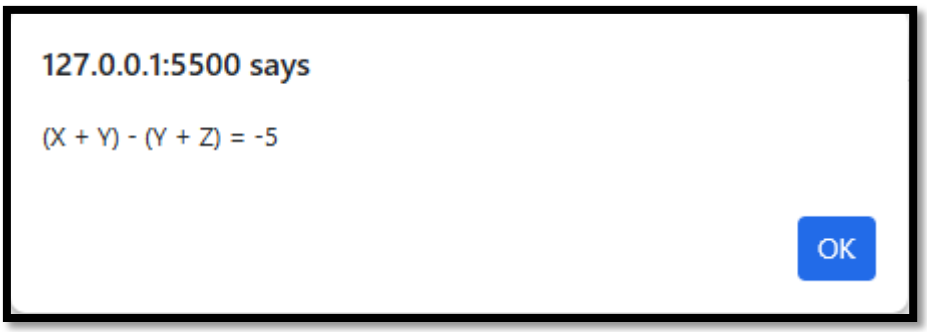
c. $X^2 + Y^2 - 2XY$

Code

```
<script>

    let x = 10;
    let y = 12;
    let z = 15;


    const sq = a => a * a;
    const add = (a, b) => a + b;
    const multi = (a, b) => a * b;
    const sub = (a, b) => a - b;
    const multis = (a, b) => 2 * a * b;


    let x2 = sq(x);
    let y2 = sq(y);


    let result1 = add(add(x2, y2), multi(x, y));
    alert("X² + Y² + XY = " + result1);


    let result2 = sub(add(x, y), add(y, z));
    alert("(X + Y) - (Y + Z) = " + result2);


    let result3 = sub(add(x2, y2), multis(x, y));
    alert("X² + Y² - 2XY = " + result3);
</script>
```

Output





8	**Write a map function for finding the square of each array elements.**
Code	

```
const numbers = [1, 2, 3, 4, 5];

const squares = numbers.map(num => num * num);

console.log("Original Array:", numbers);
console.log("Square of Elements:", squares);
```

Output

```
PS F:\Kaifclash\fullstack\labmanual> node
Original Array: [ 1, 2, 3, 4, 5 ]
Square of Elements: [ 1, 4, 9, 16, 25 ]
PS F:\Kaifclash\fullstack\labmanual>
```

9	**Write map function for finding the factorial of each elements of array.**
Code	

```
const numbers = [1, 2, 3, 4, 5];

const factorial = (n) => {
    let fact = 1;
    for (let i = 1; i <= n; i++) {
```

```
        fact *= i;
    }
    return fact;
};

// Map function
const result = numbers.map(num => factorial(num));

console.log("Original Array:", numbers);
console.log("Factorial of Elements:", result);
```

Output

```
PS F:\Kaifclash\fullstack\labmanual> node "
Original Array: [ 1, 2, 3, 4, 5 ]
Factorial of Elements: [ 1, 2, 6, 24, 120 ]
PS F:\Kaifclash\fullstack\labmanual>
```

10 **Write map function for check element is prime or not for each elements of array.**

Code

```
const numbers = [2, 3, 4, 5, 6, 7];

const isPrime = (n) => {
    if (n < 2) return false;

    for (let i = 2; i <= Math.sqrt(n); i++) {
        if (n % i === 0) {
            return false;
        }
    }
    return true;
};

// Using map
const result = numbers.map(num => isPrime(num));

console.log("Original Array:", numbers);
console.log("Prime Check:", result);
```

Output

```
PS F:\Kaifclash\fullstack\labmanual> node "f:\Kaifclash\
Original Array: [ 2, 3, 4, 5, 6, 7 ]
Prime Check: [ true, true, false, true, false, true ]
PS F:\Kaifclash\fullstack\labmanual>
```

11	**Write map function for check element is odd or even for each elements of array.**
Code	

```
const numbers = [1, 2, 3, 4, 5, 6];

const result = numbers.map(num => (num % 2 === 0 ? "Even" : "Odd"));

console.log("Original Array:", numbers);
console.log("Odd/Even Check:", result);
```

Output

```
PS F:\Kaifclash\fullstack\labmanual> node "f:\Kaifclash\fullstack\l
Original Array: [ 1, 2, 3, 4, 5, 6 ]
Odd/Even Check: [ 'Odd', 'Even', 'Odd', 'Even', 'Odd', 'Even' ]
PS F:\Kaifclash\fullstack\labmanual>
```

12	**Create studInfo.js file and write two functions as sayCollege and sayCourse. Export it and use it and show the proper message.**
Code	

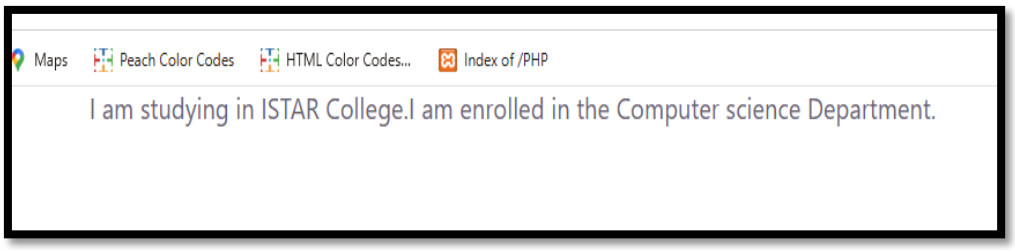
Saycollege.jsx

```
function sayCollege()
{
    return "I am studying in ISTAR College.";
}
function sayCourse()
{
    return "I am enrolled in the Computer science Department.";
}
export default  {sayCollege,sayCourse};
```

Main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    {/* <App /> */}
    <SayCollege.sayCollege/>
    <SayCollege.sayCourse/>

  </StrictMode>,
)
```

Output



13

Write a function component as Item with data as name, cost, and type. Write another function component as feedback with data as id, title, detail, and rating.
Create another function component as ItemDetail and arrange above both components in it. (Pass the data using props)

Code

Item.jsx

```
function Item(props)
{
    return(
        <table border="1">
            <tr>
                <td>Item Name:</td>
                <td>{props.name}</td>
            </tr>
            <tr>
                <td>Item Cost:</td>
                <td>{props.cost}</td>
            </tr>
            <tr>
                <td>Item type:</td>
                <td>{props.type}</td>
            </tr>
        </table>
    );
}
export default Item;
```

Feedback.jsx:-

```
function Feedback(props)
{
    return(
        <table border="1">
            <tr>
                <td>Feedback ID:</td>
                <td>{props.fid}</td>
            </tr>
            <tr>
                <td>Feedback title:</td>
                <td>{props.fdt}</td>
```

```
            </tr>
            <tr>
                <td>feedback details:</td>
                <td>{props.fdd}</td>
            </tr>
            <tr>
                <td>feedback rating:</td>
                <td>{props.fdr}</td>
            </tr>
        </table>
    )
}
export default Feedback;
```

ItemDetail.jsx

```
import Item from "./Item";
import Feedback from "./Feedback";
function ItemDetail(props)
{
    return(
        <body>
            <table>
                <tr><td colSpan={2}>Item Details</td></tr>
                <tr><td><Item name={props.name} cost={props.cost}
type={props.type}/></td></tr>
                <tr><td colSpan={2}>Feedback Details</td></tr>
                <tr><td><Feedback fid={props.fid} fdt={props.fdt}
fdd={props.fdd} fdr={props.fdr}/></td></tr>
        </table>
        </body>

    );
}
export default ItemDetail;
```

Main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>

    <Itemdetail name="abc" cost="100" type="jwellery" fid="1"
fdt="nice" fdd="nothing" fdr="5"/>
  </StrictMode>,
)
```

Output

Item Details

Item Name:	abc
Item Cost:	100
Item type:	jwellery

Feedback Details

Feedback ID:	1
Feedback title:	nice
feedback details:	nothing
feedback rating:	5

14

Write a function component as Student with data as name, rno, and city. Write another function component as Marks with data as m1, m2, m3, total, and per.
Create another function component as StudentDetail and arrange above both components in it.(Pass the data using props)

Code

Student.jsx:-

```
import React from "react";
function Student({name,rno,city})
{
    return(
        <table border="1" cellPadding="13" align="center">
            <thead>
                <tr>
                    <th colSpan="2">Student Details</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>Name:</td>
                    <td>{name}</td>
                </tr>
                <tr>
```

```
                    <td>Roll No:</td>
                    <td>{rno}</td>
                </tr>
                <tr>
                    <td>City:</td>
                    <td>{city}</td>
                </tr>
            </tbody>
        </table>
    )
}
export default Student
```

Mark.jsx:-

```
import React from "react";
function Mark({m1,m2,m3,total,per})
{
    return(
        <table border="1" cellPadding="8" align="center">
            <thead>
                <tr>
                    <th colSpan={2}>Mark Details</th>
                </tr>
            </thead>
            <tbody>
                <tr>
                    <td>Mark 1 :</td>
                    <td>{m1}</td>
                </tr>
                <tr>
                    <td>Mark 2 :</td>
                    <td>{m2}</td>
                </tr>
                <tr>
                    <td>Mark 3 :</td>
                    <td>{m3}</td>
                </tr>
                <tr>
                    <td>Total :</td>
                    <td>{total}</td>
                </tr>
                <tr>
                    <td>Percentage :</td>
                    <td>{per}</td>
                </tr>
            </tbody>
        </table>
    )
}
export default Mark
```

studentdetail.jsx:-

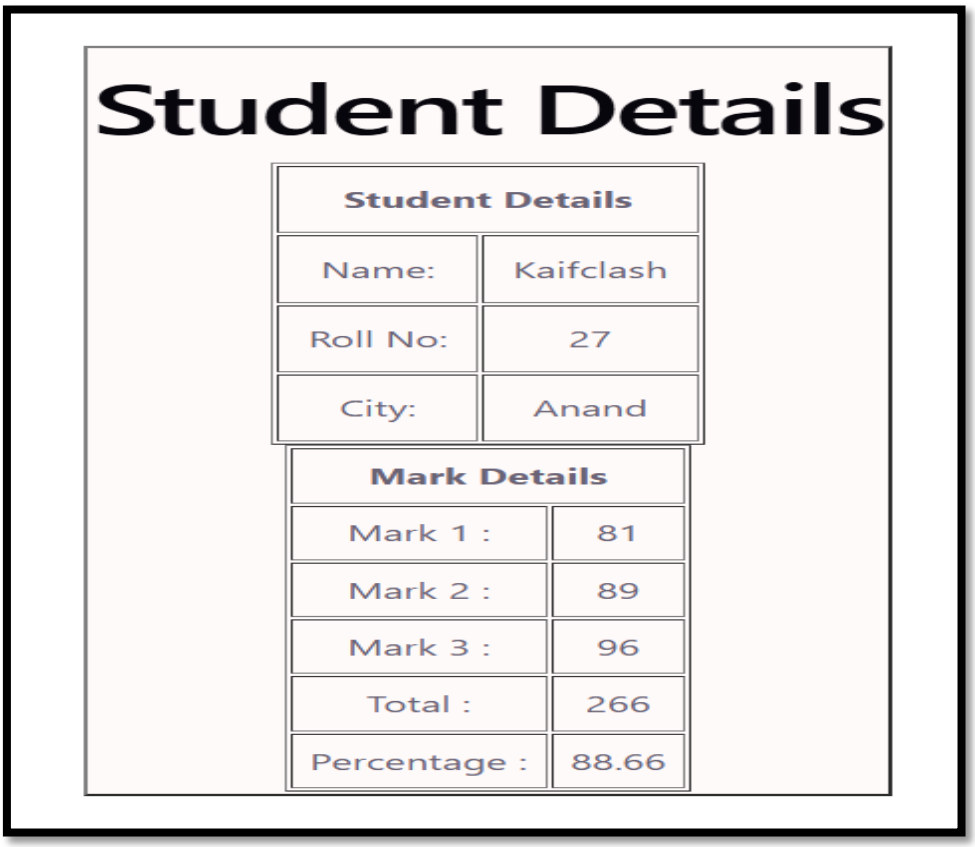
```
import Student from "./student";
import Mark from "./mark";
function StudentDetail()
```

```
{
    const studentData={name:"Dhavni" , rno : 12 , city : "Anand"};
    const markData={m1:81 , m2:89, m3:96, total:266, per:88.66};

    return(
        <table border="2" bgcolor="light-pink">
            <tr>
                <h1>Student Details</h1>
            <Student {...studentData} />
            <Mark {...markData} />
            </tr>
        </table>
    );
}
export default StudentDetail
```

main.jsx:-

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'
import StudentDetail from './lab_manual/studentdetail.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    {/* <App /> */}
    <StudentDetail/>
  </StrictMode>,
)
```

Output

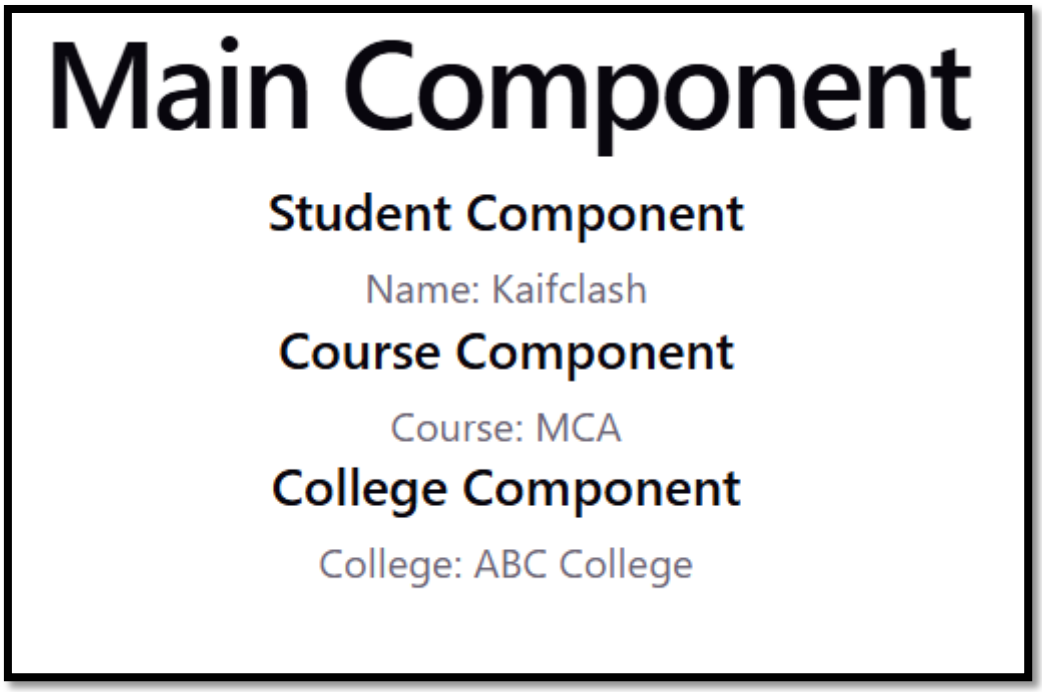


15	**Create one main function component which includes three sub function components as per your choice. (Pass the data using props)**
Code	

```
import React from "react";

function App() {
  const name = "Kaifclash";
  const course = "MCA";
  const college = "ABC College";

  return (
    <div>
      <h1>Main Component</h1>

      <Student name={name} />
      <Course course={course} />
      <College college={college} />
    </div>
  );
}

// Sub Component 1
function Student(props) {
  return (
    <div>
      <h2>Student Component</h2>
      <p>Name: {props.name}</p>
    </div>
  );
}

// Sub Component 2
function Course(props) {
  return (
    <div>
      <h2>Course Component</h2>
      <p>Course: {props.course}</p>
    </div>
  );
}

// Sub Component 3
function College(props) {
  return (
    <div>
      <h2>College Component</h2>
      <p>College: {props.college}</p>
    </div>
  );
}
```

```
export default App;
```

Output



16 **Create a class component as Bank which set States as name, branch, and ifsc. Create the element of it and show all the details.**

Code

```
import React, { Component } from "react";

class Bank extends Component {
 constructor() {
  super();

  this.state = {
   name: "State Bank of India",
   branch: "Anand",
   ifsc: "SBIN0001234"
  };
 }
 render() {
  return (
   <div>
    <h1>Bank Details</h1>
    <p>Bank Name: {this.state.name}</p>
    <p>Branch: {this.state.branch}</p>
    <p>IFSC: {this.state.ifsc}</p>
   </div>
  );
 }
}
```

Output

17

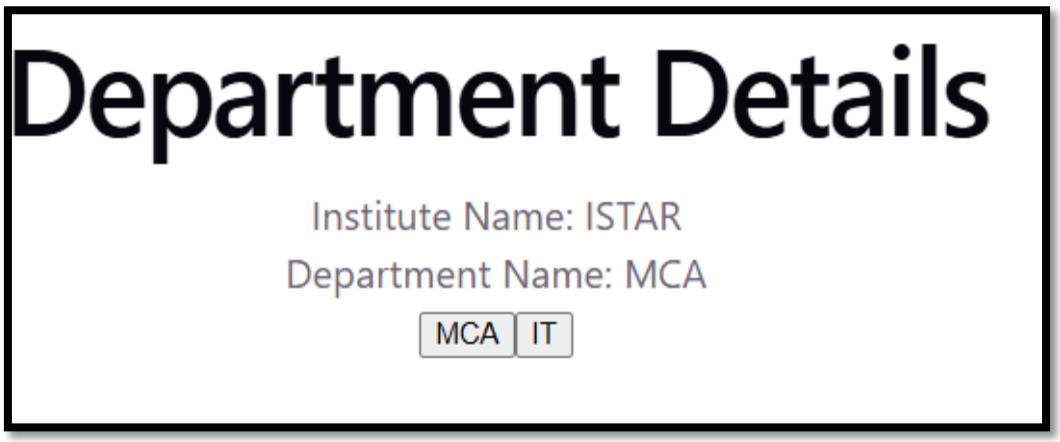
Create a class component as Department with two states as iname and dname. Assign default value as ISTAR and blank. Take two buttons MCA and IT. Set the state (iname and dname) according to the click on button.

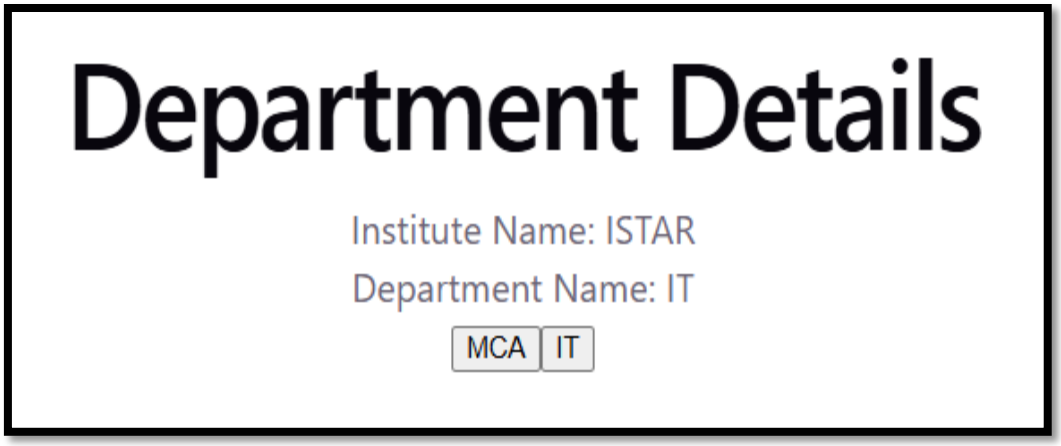
Code

```
import React, { Component } from "react";

class Department extends Component {
 constructor() {
  super();

  this.state = {
   iname: "ISTAR",
   dname: ""
  };
 }

 setMCA = () => {
  this.setState({
   iname: "ISTAR",
   dname: "MCA"
  });
 };

 setIT = () => {
  this.setState({
   iname: "ISTAR",
   dname: "IT"
  });
 };

 render() {
  return (
   <div>
    <h1>Department Details</h1>

    <p>Institute Name: {this.state.iname}</p>
```

```
      <p>Department Name: {this.state.dname}</p>

      <button onClick={this.setMCA}>MCA</button>
      <button onClick={this.setIT}>IT</button>
    </div>
  );
 }
}

function App() {
 return (
  <div>
   <Department />
  </div>
 );
}

export default App;
```

Output





18 **Create class component as StateDetail with data as sname, capital, area, language, Dance. Set a link for at least five states and show the above information (in proper format) according to the click on link.**

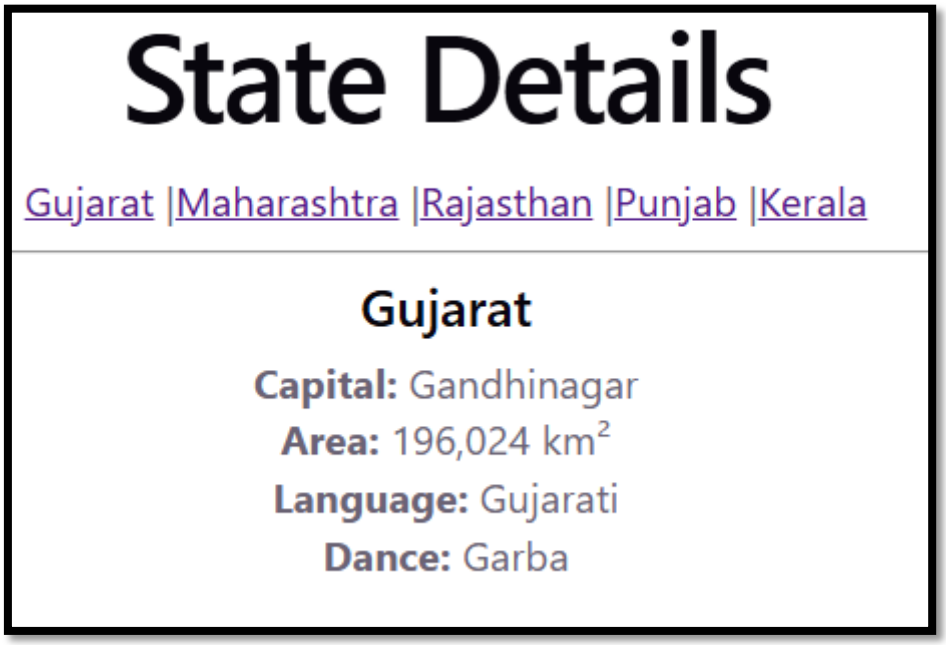
Code

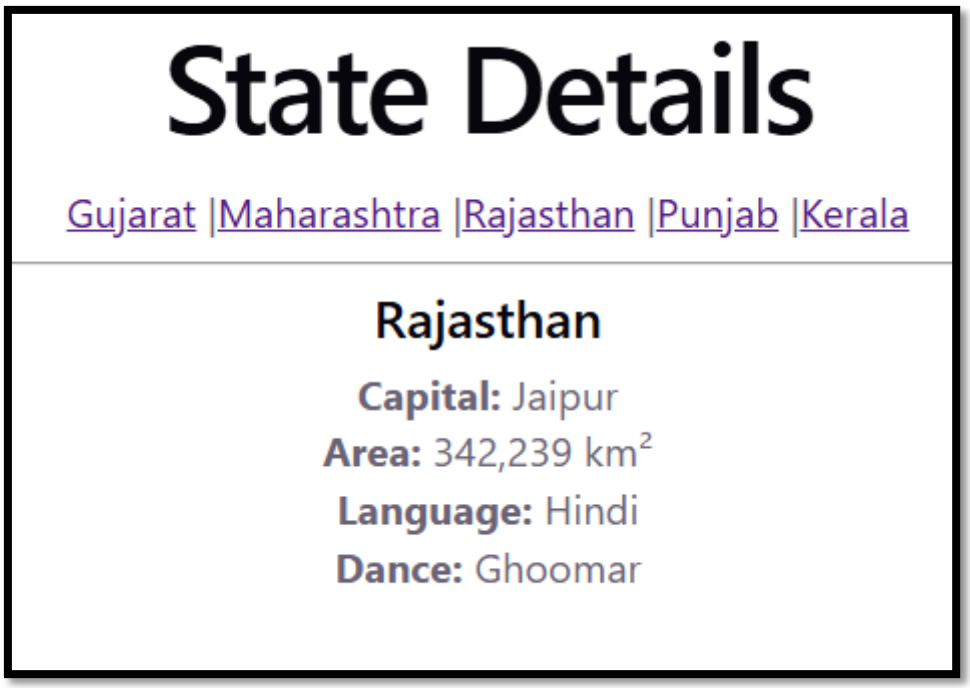
```
import React, { Component } from "react";
```

```
class StateDetail extends Component {
 constructor() {
  super();

  this.state = {
   sname: "",
   capital: "",
   area: "",
   language: "",
   dance: ""
  };
 }

 showState = (data) => {
  this.setState(data);
 };

 render() {
  return (
   <div>
    <h1>State Details</h1>

    <a href="#" onClick={() => this.showState({
     sname: "Gujarat",
     capital: "Gandhinagar",
     area: "196,024 km²",
     language: "Gujarati",
     dance: "Garba"
    })}>Gujarat</a> |

    <a href="#" onClick={() => this.showState({
     sname: "Maharashtra",
     capital: "Mumbai",
     area: "307,713 km²",
     language: "Marathi",
     dance: "Lavani"
    })}>Maharashtra</a> |

    <a href="#" onClick={() => this.showState({
     sname: "Rajasthan",
     capital: "Jaipur",
     area: "342,239 km²",
     language: "Hindi",
     dance: "Ghoomar"
    })}>Rajasthan</a> |

    <a href="#" onClick={() => this.showState({
     sname: "Punjab",
     capital: "Chandigarh",
```

```
        area: "50,362 km²",
        language: "Punjabi",
        dance: "Bhangra"
      })}>Punjab</a> |

      <a href="#" onClick={() => this.showState({
        sname: "Kerala",
        capital: "Thiruvananthapuram",
        area: "38,863 km²",
        language: "Malayalam",
        dance: "Kathakali"
      })}>Kerala</a>

      <hr />

      <h2>{this.state.sname}</h2>
      <p><b>Capital:</b> {this.state.capital}</p>
      <p><b>Area:</b> {this.state.area}</p>
      <p><b>Language:</b> {this.state.language}</p>
      <p><b>Dance:</b> {this.state.dance}</p>
    </div>
  );
 }
}

function App() {
  return <StateDetail />;
}

export default App;
```

Output





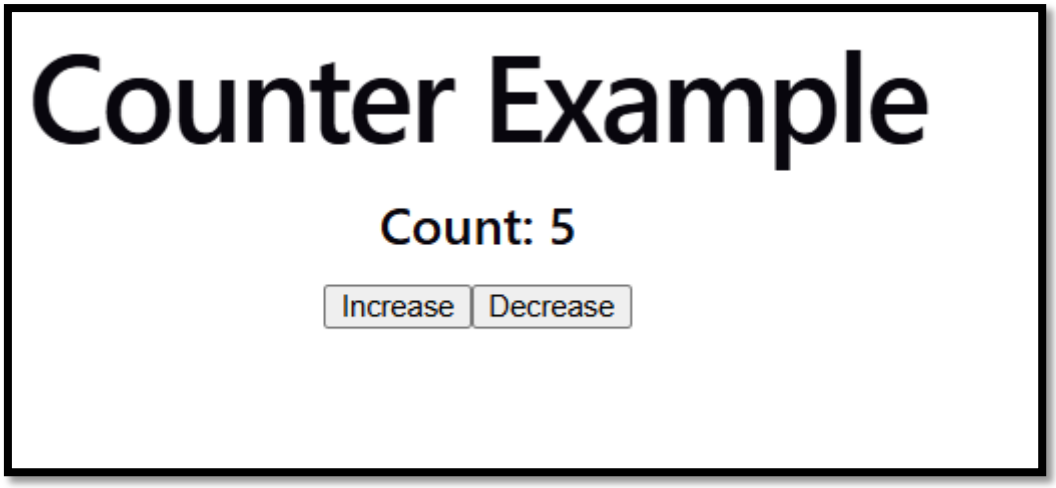
19	**Create your own class component with change the states effect.**

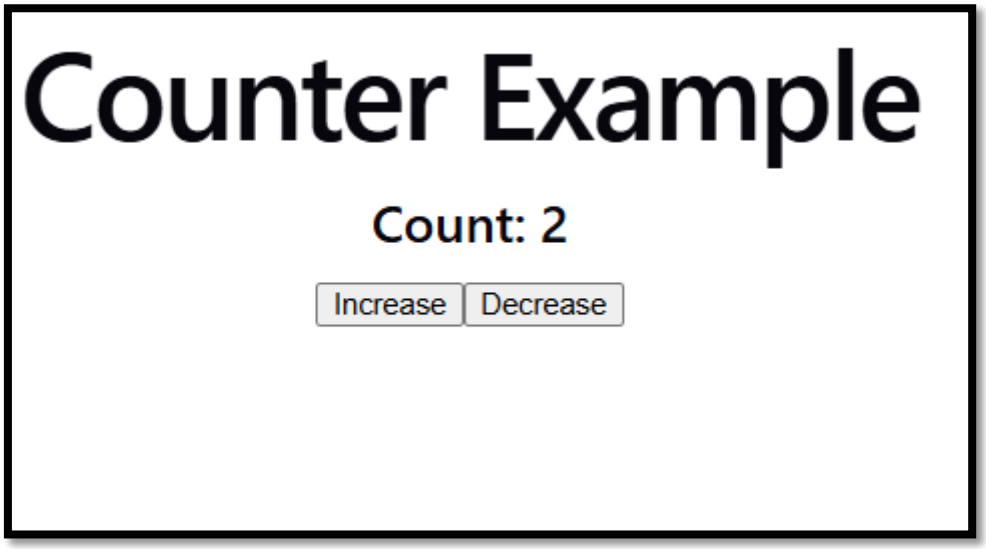
Code

```
import React, { Component } from "react";

class Counter extends Component {
 constructor() {
  super();

  this.state = {
   count: 0
  };
 }

 increment = () => {
  this.setState({ count: this.state.count + 1 });
 };

 decrement = () => {
  this.setState({ count: this.state.count - 1 });
 };

 render() {
  return (
   <div>
    <h1>Counter Example</h1>

    <h2>Count: {this.state.count}</h2>

    <button onClick={this.increment}>Increase</button>
    <button onClick={this.decrement}>Decrease</button>
```

```
      </div>
    );
  }
}

function App() {
  return <Counter />;
}

export default App;
```

Output





20

Create function components as home, about, contact and NoPage. Create function component for layout. Create multi page application for these componets using Router.

Code

```
import React from "react";
import { BrowserRouter, Routes, Route, Link, Outlet } from "react-router-dom";

// Layout
function Layout() {
  return (
```

```
    <div>
      <h1>My Website</h1>

      <nav>
        <Link to="/">Home</Link> |
        <Link to="/about">About</Link> |
        <Link to="/contact">Contact</Link>
      </nav>

      <hr />
      <Outlet />
    </div>
  );
}

// Pages
function Home() {
  return <h2>Home Page</h2>;
}

function About() {
  return <h2>About Page</h2>;
}

function Contact() {
  return <h2>Contact Page</h2>;
}

function NoPage() {
  return <h2>404 - Page Not Found</h2>;
}

// App
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="about" element={<About />} />
          <Route path="contact" element={<Contact />} />
          <Route path="*" element={<NoPage />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Output	
21	**Create class components as India, Australia, SouthAfrica, England and NewZeeland. Each component have at least five players link which show information according to clicked on it. The information of the player will be name, matches, run, 50s, 100s, and wickets. Set Layout component and Routes for each country using inbuilt Route package.**
Code	

```
import React, { Component } from "react";
import { BrowserRouter, Routes, Route, Link, Outlet } from "react-router-
dom";

/* ================= LAYOUT ================= */
function Layout() {
  return (
    <div>
      <h1>Cricket Teams App</h1>

      <nav>
```

```
      <Link to="/india">India</Link> |
      <Link to="/australia">Australia</Link> |
      <Link to="/southafrica">South Africa</Link> |
      <Link to="/england">England</Link> |
      <Link to="/newzeeland">New Zealand</Link>
    </nav>

    <hr />
    <Outlet />
  </div>
 );
}

/* ================= INDIA ================= */
class India extends Component {
 state = { player: {} };

 render() {
  const show = (p) => this.setState({ player: p });

  return (
    <div>
      <h2>India Team</h2>

      <button onClick={() => show({
        name: "Virat Kohli",
        matches: 274,
        runs: 12898,
        fifties: 65,
        hundreds: 46,
        wickets: 4
      })}>Virat Kohli</button>

      <button onClick={() => show({
        name: "Rohit Sharma",
        matches: 262,
        runs: 10709,
        fifties: 55,
        hundreds: 31,
        wickets: 8
      })}>Rohit Sharma</button>

      <hr />

      <h3>{this.state.player.name}</h3>
      <p>Matches: {this.state.player.matches}</p>
      <p>Runs: {this.state.player.runs}</p>
      <p>50s: {this.state.player.fifties}</p>
      <p>100s: {this.state.player.hundreds}</p>
```

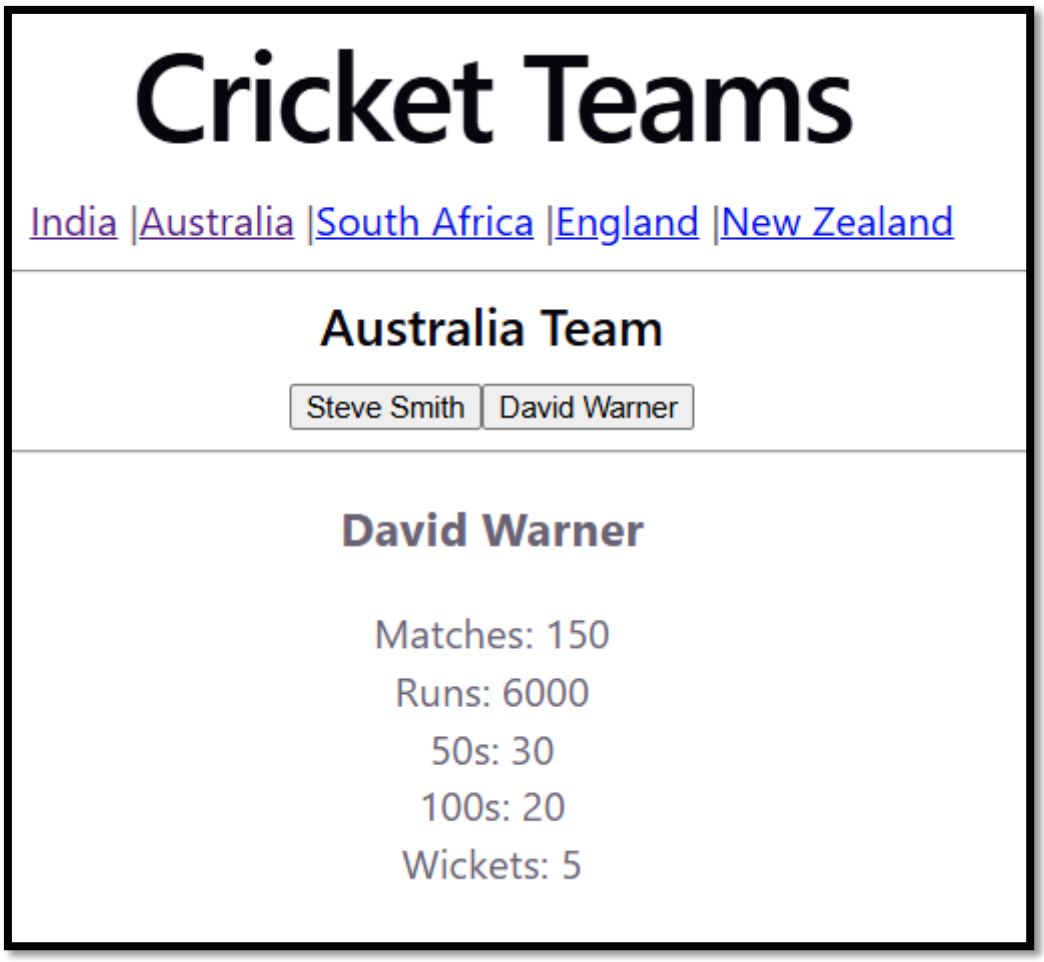
```
      <p>Wickets: {this.state.player.wickets}</p>
    </div>
  );
 }
}

/* ================= AUSTRALIA ================= */
class Australia extends Component {
 state = { player: {} };

 render() {
  const show = (p) => this.setState({ player: p });

  return (
    <div>
      <h2>Australia Team</h2>

      <button onClick={() => show({
        name: "Steve Smith",
        matches: 170,
        runs: 8000,
        fifties: 37,
        hundreds: 30,
        wickets: 17
      })}>Steve Smith</button>

      <button onClick={() => show({
        name: "David Warner",
        matches: 150,
        runs: 6000,
        fifties: 30,
        hundreds: 20,
        wickets: 5
      })}>David Warner</button>

      <hr />

      <h3>{this.state.player.name}</h3>
      <p>Matches: {this.state.player.matches}</p>
      <p>Runs: {this.state.player.runs}</p>
      <p>50s: {this.state.player.fifties}</p>
      <p>100s: {this.state.player.hundreds}</p>
      <p>Wickets: {this.state.player.wickets}</p>
    </div>
  );
 }
}

/* ================= SOUTH AFRICA ================= */
```

```
class SouthAfrica extends Component {
  state = { player: {} };

  render() {
    const show = (p) => this.setState({ player: p });

    return (
      <div>
        <h2>South Africa Team</h2>

        <button onClick={() => show({
          name: "AB de Villiers",
          matches: 228,
          runs: 9577,
          fifties: 53,
          hundreds: 25,
          wickets: 5
        })}>AB de Villiers</button>

        <hr />

        <h3>{this.state.player.name}</h3>
        <p>Matches: {this.state.player.matches}</p>
        <p>Runs: {this.state.player.runs}</p>
        <p>50s: {this.state.player.fifties}</p>
        <p>100s: {this.state.player.hundreds}</p>
        <p>Wickets: {this.state.player.wickets}</p>
      </div>
    );
  }
}

/* ================= ENGLAND ================= */
class England extends Component {
  state = { player: {} };

  render() {
    const show = (p) => this.setState({ player: p });

    return (
      <div>
        <h2>England Team</h2>

        <button onClick={() => show({
          name: "Joe Root",
          matches: 150,
          runs: 9000,
          fifties: 50,
          hundreds: 30,
```

```
        wickets: 10
      })}>Joe Root</button>

      <hr />

      <h3>{this.state.player.name}</h3>
      <p>Matches: {this.state.player.matches}</p>
      <p>Runs: {this.state.player.runs}</p>
      <p>50s: {this.state.player.fifties}</p>
      <p>100s: {this.state.player.hundreds}</p>
      <p>Wickets: {this.state.player.wickets}</p>
    </div>
  );
 }
}

/* ================= NEW ZEALAND ================= */
class NewZeeland extends Component {
 state = { player: {} };

 render() {
  const show = (p) => this.setState({ player: p });

  return (
    <div>
      <h2>New Zealand Team</h2>

      <button onClick={() => show({
        name: "Kane Williamson",
        matches: 160,
        runs: 7000,
        fifties: 40,
        hundreds: 24,
        wickets: 5
      })}>Kane Williamson</button>

      <hr />

      <h3>{this.state.player.name}</h3>
      <p>Matches: {this.state.player.matches}</p>
      <p>Runs: {this.state.player.runs}</p>
      <p>50s: {this.state.player.fifties}</p>
      <p>100s: {this.state.player.hundreds}</p>
      <p>Wickets: {this.state.player.wickets}</p>
    </div>
  );
 }
}
```

```
/* ================= APP ROUTES ================= */
function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route path="india" element={<India />} />
          <Route path="australia" element={<Australia />} />
          <Route path="southafrica" element={<SouthAfrica />} />
          <Route path="england" element={<England />} />
          <Route path="newzeeland" element={<NewZeeland />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}

export default App;
```

Output



22	**Think about one real-life practical multi-page application as per your perception and implement it using both class and Routes (similar like prog 21).**
Code	

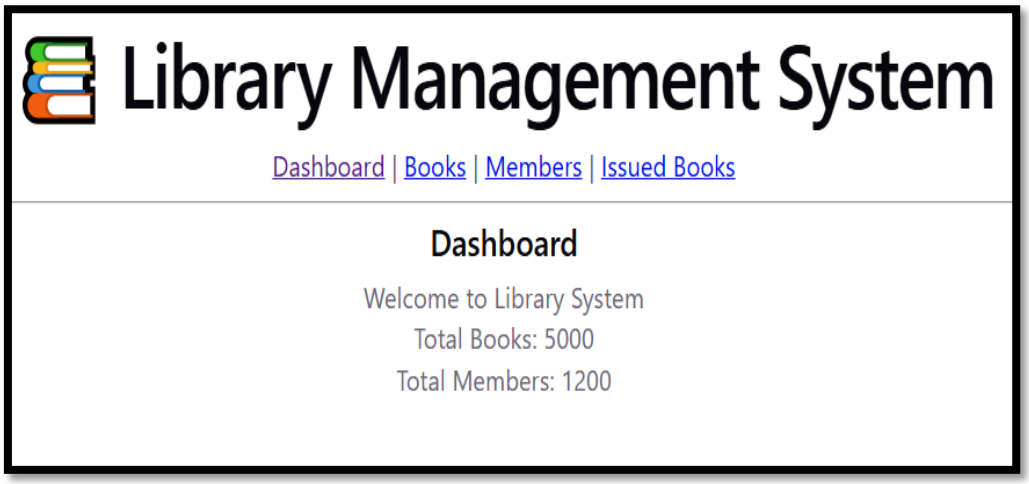
```
import React, { Component } from "react";
import { BrowserRouter, Routes, Route, Link, Outlet } from "react-router-dom";

/* ================= LAYOUT ================= */
function Layout() {
 return (
  <div>
   <h1>   Library Management System</h1>

   <nav>
    <Link to="/dashboard">Dashboard</Link> |{" "}
    <Link to="/books">Books</Link> |{" "}
    <Link to="/members">Members</Link> |{" "}
    <Link to="/issued">Issued Books</Link>
   </nav>

   <hr />
   <Outlet />
```

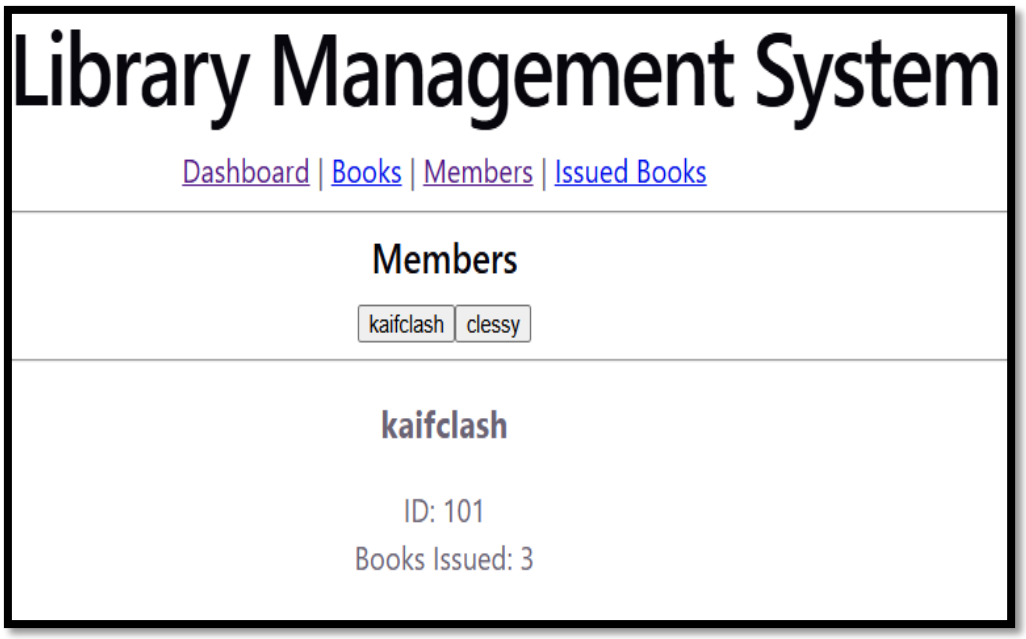
```
    </div>
  );
}

/* ================= DASHBOARD ================= */
class Dashboard extends Component {
  render() {
    return (
      <div>
        <h2>Dashboard</h2>
        <p>Welcome to Library System</p>
        <p>Total Books: 5000</p>
        <p>Total Members: 1200</p>
      </div>
    );
  }
}

/* ================= BOOKS ================= */
class Books extends Component {
  state = { book: {} };

  showBook = (b) => {
    this.setState({ book: b });
  };

  render() {
    return (
      <div>
        <h2>Books</h2>

        <button onClick={() => this.showBook({
          title: "Harry Potter",
          author: "J.K. Rowling",
          pages: 400,
          year: 1997
        })}>
          Harry Potter
        </button>

        <button onClick={() => this.showBook({
          title: "The Alchemist",
          author: "Paulo Coelho",
          pages: 208,
          year: 1988
        })}>
          The Alchemist
        </button>
```

```
      <hr />

      <h3>{this.state.book.title}</h3>
      <p>Author: {this.state.book.author}</p>
      <p>Pages: {this.state.book.pages}</p>
      <p>Year: {this.state.book.year}</p>
    </div>
  );
 }
}

/* ================= MEMBERS ================= */
class Members extends Component {
 state = { member: {} };

 showMember = (m) => {
  this.setState({ member: m });
 };

 render() {
  return (
    <div>
      <h2>Members</h2>

      <button onClick={() => this.showMember({
        name: "Amit Patel",
        id: 101,
        booksIssued: 3
      })}>
        Amit Patel
      </button>

      <button onClick={() => this.showMember({
        name: "Sneha Sharma",
        id: 102,
        booksIssued: 5
      })}>
        Sneha Sharma
      </button>

      <hr />

      <h3>{this.state.member.name}</h3>
      <p>ID: {this.state.member.id}</p>
      <p>Books Issued: {this.state.member.booksIssued}</p>
    </div>
  );
 }
}
```

```
/* ================= ISSUED BOOKS ================= */
class Issued extends Component {
 render() {
  return (
   <div>
    <h2>Issued Books</h2>
    <ul>
     <li>Harry Potter → Amit Patel</li>
     <li>The Alchemist → Sneha Sharma</li>
     <li>Wings of Fire → Raj Mehta</li>
    </ul>
   </div>
  );
 }
}

/* ================= APP ROUTES ================= */
function App() {
 return (
  <BrowserRouter>
   <Routes>
    <Route path="/" element={<Layout />}>
     <Route path="dashboard" element={<Dashboard />} />
     <Route path="books" element={<Books />} />
     <Route path="members" element={<Members />} />
     <Route path="issued" element={<Issued />} />
    </Route>
   </Routes>
  </BrowserRouter>
 );
}

export default App;
```

Output



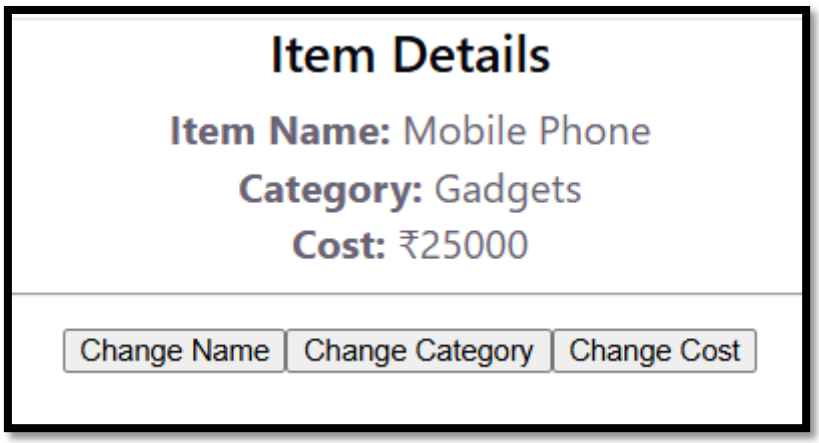


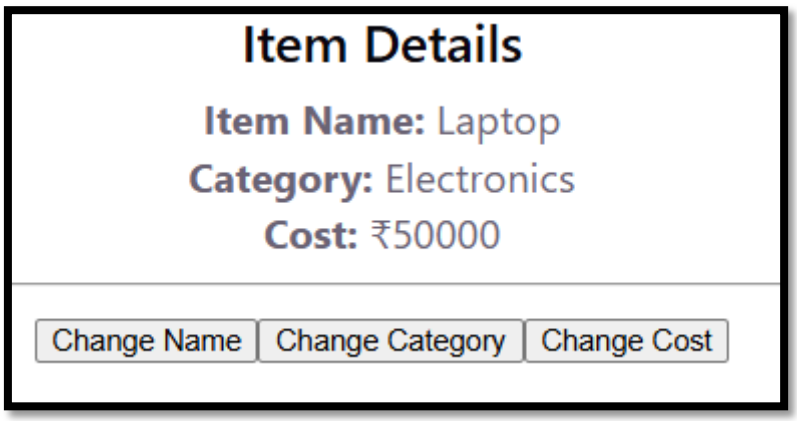
23	**Create function component as Item. Write different useState to initialize and change the data like iname, caegory, and cost.**
Code	

```
import React, { useState } from "react";

function Item() {
  // useState for item name
  const [iname, setIname] = useState("Laptop");

  // useState for category
  const [category, setCategory] = useState("Electronics");

  // useState for cost
  const [cost, setCost] = useState(50000);

  return (
    <div>
      <h2>Item Details</h2>

      <p><b>Item Name:</b> {iname}</p>
      <p><b>Category:</b> {category}</p>
      <p><b>Cost:</b> ₹{cost}</p>

      <hr />

      {/* Buttons to change state */}
      <button onClick={() => setIname("Mobile Phone")}>
        Change Name
      </button>

      <button onClick={() => setCategory("Gadgets")}>
        Change Category
      </button>
```

```
    <button onClick={() => setCost(25000)}>
      Change Cost
    </button>
  </div>
 );
}

export default Item;
```

Output





24

Create function component as Pizza. Write one function as setPizze using useState to initialize and change the data like name, type (small, medium, large), toppings, and cost. The data will be change when user click on different buttons like (Margherita, Farmhouse, Fresh Veggie, Paneer Makhani, Cheesy7). Crate functions for each button like setMargherita for Margherita button. Call the setPizza function inside the setMargherita and set the data(states) as per the category of pizza.

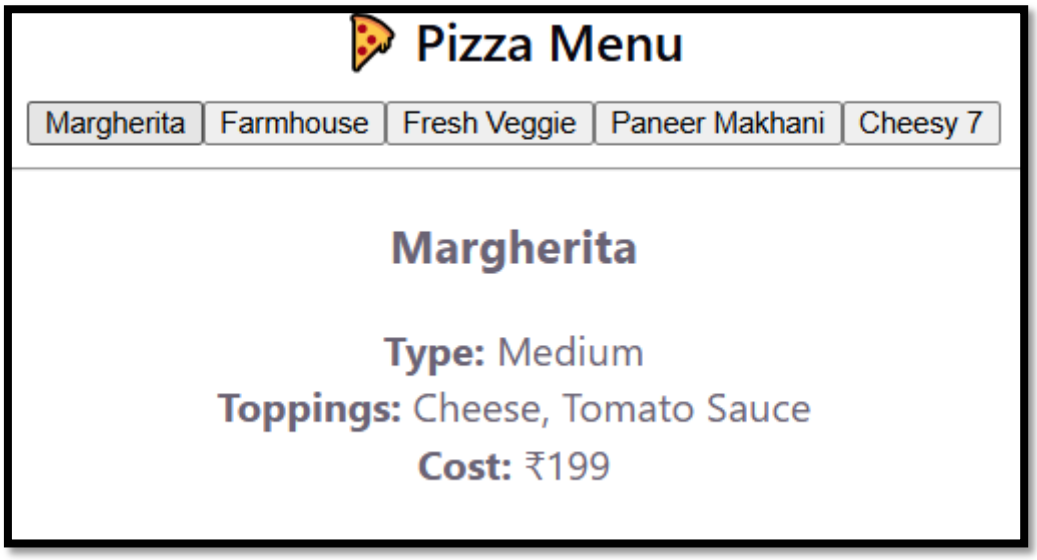
Code

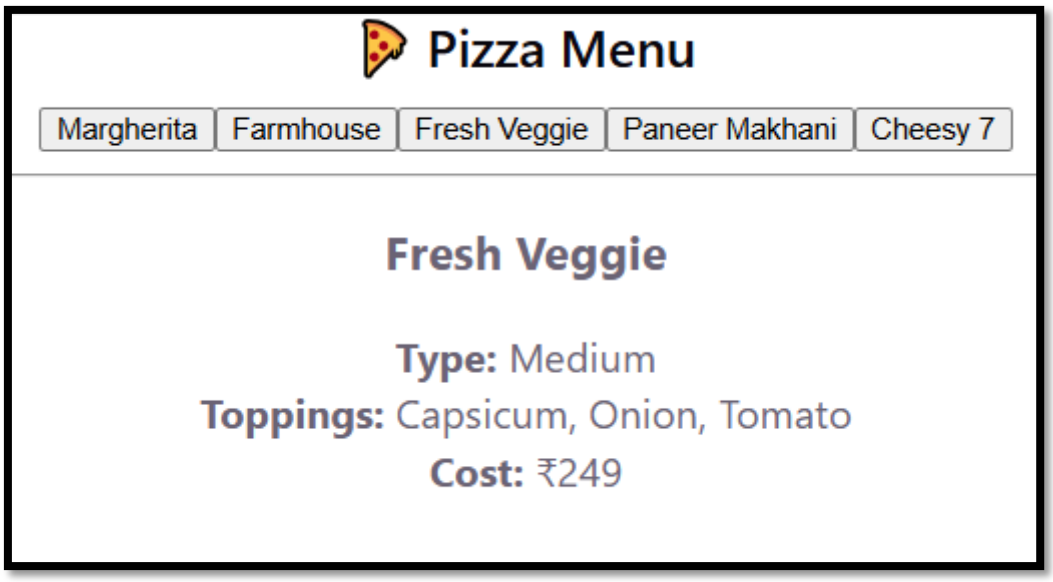
```
import React, { useState } from "react";

function Pizza() {
 // single state object
 const [pizza, setPizza] = useState({
   name: "Select Pizza",
   type: "",
   toppings: [],
   cost: 0
 });
```

```
//    Margherita
const setMargherita = () => {
  setPizza({
    name: "Margherita",
    type: "Medium",
    toppings: ["Cheese", "Tomato Sauce"],
    cost: 199
  });
};

//    Farmhouse
const setFarmhouse = () => {
  setPizza({
    name: "Farmhouse",
    type: "Large",
    toppings: ["Onion", "Capsicum", "Tomato", "Corn"],
    cost: 299
  });
};

//    Fresh Veggie
const setFreshVeggie = () => {
  setPizza({
    name: "Fresh Veggie",
    type: "Medium",
    toppings: ["Capsicum", "Onion", "Tomato"],
    cost: 249
  });
};

//    Paneer Makhani
const setPaneerMakhani = () => {
  setPizza({
    name: "Paneer Makhani",
    type: "Large",
    toppings: ["Paneer", "Makhani Sauce", "Onion"],
    cost: 349
  });
};

//    Cheesy 7
const setCheesy7 = () => {
  setPizza({
    name: "Cheesy 7",
    type: "Large",
    toppings: [
      "Mozzarella",
      "Cheddar",
```

```
      "Parmesan",
      "Cheese Burst",
      "Corn",
      "Capsicum",
      "Jalapeno"
    ],
    cost: 399
  });
};

return (
  <div>
    <h2>    Pizza Menu</h2>

    {/* Buttons */}
    <button onClick={setMargherita}>Margherita</button>
    <button onClick={setFarmhouse}>Farmhouse</button>
    <button onClick={setFreshVeggie}>Fresh Veggie</button>
    <button onClick={setPaneerMakhani}>Paneer Makhani</button>
    <button onClick={setCheesy7}>Cheesy 7</button>

    <hr />

    {/* Display Data */}
    <h3>{pizza.name}</h3>
    <p><b>Type:</b> {pizza.type}</p>
    <p><b>Toppings:</b> {pizza.toppings.join(", ")}</p>
    <p><b>Cost:</b> ₹{pizza.cost}</p>
  </div>
);
}

export default Pizza;
```

Output





25 **Write react code hor reading the json file and set the content of it into react component.**

Code

Data.json

```
[
  {
    "id": 1,
    "name": "Apple",
    "category": "Fruit",
    "price": 120
  },
  {
    "id": 2,
    "name": "Banana",
    "category": "Fruit",
    "price": 40
  },
  {
    "id": 3,
    "name": "Carrot",
    "category": "Vegetable",
    "price": 60
  }
]
```

Items.jsx

```
import React, { useState, useEffect } from "react";
import data from "./data.json";

function Items() {
  const [items, setItems] = useState([]);

  // Load JSON data into state
```

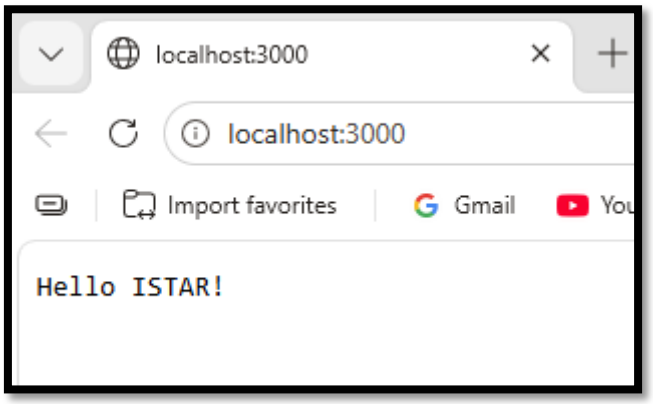
```
  useEffect(() => {
    setItems(data);
  }, []);

  return (
    <div>
      <h2>Items List</h2>

      {items.map((item) => (
        <div key={item.id} style={{ border: "1px solid black", margin: "10px",
padding: "10px" }}>
          <h3>{item.name}</h3>
          <p>Category: {item.category}</p>
          <p>Price: ₹{item.price}</p>
        </div>
      ))}
    </div>
  );
}

export default Items;
```

Output

NodeJs

1	**Create a first.js file which creates server and show the message "Hello ISTAR!" as response.**
Code	

```
const http = require("http");

// Create server
const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello ISTAR!");
});

// Server listening on port 3000
server.listen(3000, () => {
  console.log("Server is running on http://localhost:3000");
});
```

Output	
2	**Create one user-define module as say.js which contains four methods like sayCollege() which returns "Hello ISTAR" and sayDepartment() which returns "Hello MCA". Include this module in main.js file and use its functionalities.**

Code	**<u>Say.js</u>**
	function sayCollege() {
	  return "Hello ISTAR";
	}
	
	function sayDepartment() {
	  return "Hello MCA";
	}
	
	function sayName(name) {
	  return "Hello " + name;
	}
	
	function sayWelcome() {
	  return "Welcome to Node.js Module System";
	}
	
	// Export functions
	module.exports = {
	  sayCollege,
	  sayDepartment,
	  sayName,
	  sayWelcome
	};
	
	<u>Main.js</u>
	
	`const say = require("./say");`
	
	`// Using module functions`
	`console.log(say.sayCollege());`
	`console.log(say.sayDepartment());`

```
console.log(say.sayName("Student"));
console.log(say.sayWelcome());
```

Output

```
F:\Kaifclash\fullstack\Node>node main.js
Hello ISTAR
Hello MCA
Hello Student
Welcome to Node.js Module System
```

3

Create one user-define module as arith.js which contains methods like add(x,y), sub(x,y), mul(x,y), div(x,y), fact(x), square(x), sqroot(x). Include this module in main.js and use its functionalities by passing the arguments as per prototype of method.

Code

```
function add(x, y) {
  return x + y;
}

function sub(x, y) {
  return x - y;
}

function mul(x, y) {
  return x * y;
}

function div(x, y) {
  if (y === 0) return "Cannot divide by zero";
  return x / y;
}

function fact(x) {
  let f = 1;
  for (let i = 1; i <= x; i++) {
    f *= i;
  }
  return f;
}

function square(x) {
  return x * x;
}

function sqroot(x) {
  return Math.sqrt(x);
}
```

```
// export all functions
module.exports = {
  add,
  sub,
  mul,
  div,
  fact,
  square,
  sqroot
};
const arith = require("./arith");

// Using functions
console.log("Addition:", arith.add(10, 5));
console.log("Subtraction:", arith.sub(10, 5));
console.log("Multiplication:", arith.mul(10, 5));
console.log("Division:", arith.div(10, 5));

console.log("Factorial:", arith.fact(5));
console.log("Square:", arith.square(6));
console.log("Square Root:", arith.sqroot(36));
```

Output

```
F:\Kaifclash\fullstack\Node>node main2.js
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2
Factorial: 120
Square: 36
Square Root: 6
```

4

Create one user-define module as string_edit.js which contains methods like uc(x), lc(x), len(x). Include this module in main.js and use its functionalities by passing the arguments as per prototype of method.

Code

<u>string_edit.js</u>

```
function uc(x) {
  return x.toUpperCase();
}

function lc(x) {
  return x.toLowerCase();
}

function len(x) {
```

```
  return x.length;
}

// export functions
module.exports = {
  uc,
  lc,
  len
};
```

Main.js

```
const strEdit = require("./string_edit");

// Using functions
let text = "Hello ISTAR MCA";

console.log("Original:", text);
console.log("Upper Case:", strEdit.uc(text));
console.log("Lower Case:", strEdit.lc(text));
console.log("Length:", strEdit.len(text));
```

Output

```
F:\Kaifclash\fullstack\Node>node main23.js
Original: Hello ISTAR MCA
Upper Case: HELLO ISTAR MCA
Lower Case: hello istar mca
Length: 15
```

5

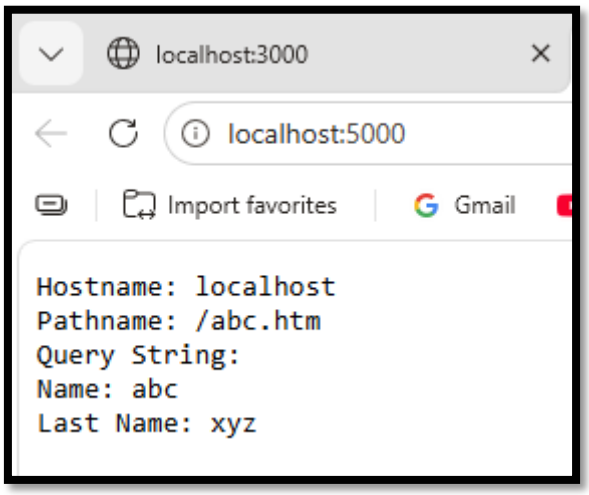
Create url_demo.js which extract the all information like host name, path name, query string from the url and display as response. url: http://localhost:5000/abc.htm?name=abc&lname=xyz

Code

```
const http = require("http");
const url = require("url");

// Create server
const server = http.createServer((req, res) => {

  // Sample URL (as given in question)
  const myURL = "http://localhost:5000/abc.htm?name=abc&lname=xyz";

  // Parse URL
  const parsedURL = url.parse(myURL, true);

  const hostname = parsedURL.hostname;
  const pathname = parsedURL.pathname;
  const query = parsedURL.query;

  // Response
```

```
  res.writeHead(200, { "Content-Type": "text/plain" });

  res.write("Hostname: " + hostname + "\n");
  res.write("Pathname: " + pathname + "\n");
  res.write("Query String:\n");
  res.write("Name: " + query.name + "\n");
  res.write("Last Name: " + query.lname + "\n");

  res.end();
});

// Server running
server.listen(5000, () => {
  console.log("Server running at http://localhost:5000");
});
```

Output



6 **Create fs_demo.js which will read the content from the test.txt and write it in new_test.txt.**

Code

<u>Fs_demo.js</u>

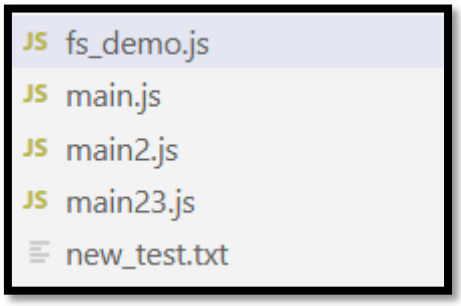
```javascript
const fs = require("fs");

// Read file
fs.readFile("test.txt", "utf8", (err, data) => {
  if (err) {
    console.log("Error reading file:", err);
    return;
  }

  console.log("File read successfully!");

  // Write into new file
  fs.writeFile("new_test.txt", data, (err) => {
    if (err) {
      console.log("Error writing file:", err);
      return;
```

```
    }

    console.log("Data written to new_test.txt successfully!");
  });
});
```

Text.txt
Hello ISTAR MCA Students!
This is original file content.

Output

```
F:\Kaifclash\fullstack\Node>node fs_demo.js
File read successfully!
Data written to new_test.txt successfully!
```



7 **Create two files as mca.html and it.html. Create one department.js file which reads the request from url and display the content of the file according it. 8. For example: if user request like http://localhost:5000/mca then it will read the mca.html file and showing the content in its response.**

Code

Mca.html

```
<!DOCTYPE html>
<html>
<head>
 <title>MCA Department</title>
</head>
<body>
 <h1>Welcome to MCA Department</h1>
 <p>This is MCA HTML page content.</p>
</body>
</html>
```

It.html

```
<!DOCTYPE html>
<html>
<head>
 <title>IT Department</title>
</head>
<body>
```

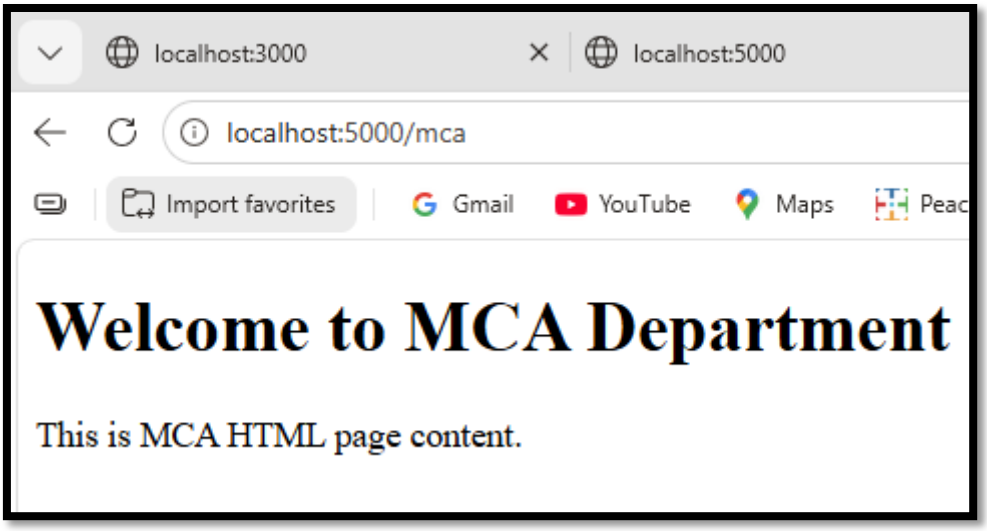
```
  <h1>Welcome to IT Department</h1>
  <p>This is IT HTML page content.</p>
</body>
</html>
```

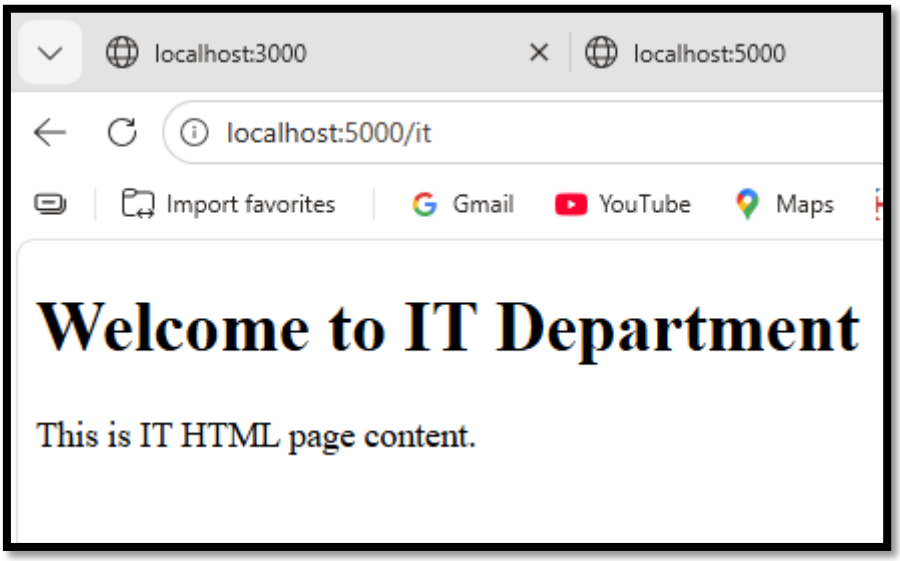
department.js

```
const http = require("http");
const fs = require("fs");

const server = http.createServer((req, res) => {

  // Extract path from URL
  let path = req.url;

  // Remove leading "/"
  if (path === "/") {
    res.end("Please enter /mca or /it");
    return;
  }

  let fileName = "";

  // Route handling
  if (path === "/mca") {
    fileName = "mca.html";
  }
  else if (path === "/it") {
    fileName = "it.html";
  }
  else {
    res.writeHead(404, { "Content-Type": "text/plain" });
    res.end("Page Not Found");
    return;
  }

  // Read file
  fs.readFile(fileName, "utf8", (err, data) => {
    if (err) {
      res.writeHead(500, { "Content-Type": "text/plain" });
      res.end("Error reading file");
      return;
    }

    // Send response
    res.writeHead(200, { "Content-Type": "text/html" });
    res.end(data);
  });
```

```
});

// Server start
server.listen(5000, () => {
  console.log("Server running at http://localhost:5000");
});
```

Output





8

Do as directed: a. Create database as "data" b. Create table as movie with the fields as mid, mname, category (Hollywood, Bollywood, Gujarati), etc. c. Insert at least the three records for each category. Retrieve the data category wise and store it into separate JSON file like Hollywood.json, Bollywood.json, and Gujarati.json.

Code

```
const mysql = require('mysql2');
const fs = require('fs');

const connection = mysql.createConnection({
```

```
  host: 'localhost',
  user: 'root',
  password: '',
  multipleStatements: true
});

connection.connect(err => {
  if (err) throw err;
  console.log("Connected to MySQL");

  const createDBAndTable = `
    CREATE DATABASE IF NOT EXISTS data;
    USE data;

    CREATE TABLE IF NOT EXISTS movie (
      mid INT AUTO_INCREMENT PRIMARY KEY,
      mname VARCHAR(255),
      category VARCHAR(50)
    );
  `;

  connection.query(createDBAndTable, (err) => {
    if (err) throw err;

    console.log("Database and table ready.");

    const insertMovies = `
      INSERT INTO movie (mname, category) VALUES
        ('Avengers', 'Hollywood'),
        ('Inception', 'Hollywood'),
        ('Titanic', 'Hollywood'),
        ('3 Idiots', 'Bollywood'),
        ('Dangal', 'Bollywood'),
        ('Sholay', 'Bollywood'),
        ('Hellaro', 'Gujarati'),
        ('Chhello Divas', 'Gujarati'),
        ('Reva', 'Gujarati');
    `;

    connection.query(insertMovies, (err) => {
      if (err) throw err;

      console.log("Movies inserted.");

      const categories = ['Hollywood', 'Bollywood', 'Gujarati'];

      let completed = 0;
```

```
      categories.forEach(category => {
        const query = `SELECT * FROM movie WHERE category = ?`;

        connection.query(query, [category], (err, results) => {
          if (err) throw err;

          fs.writeFileSync(`${category}.json`,
JSON.stringify(results, null, 2));
          console.log(`${category}.json created`);

          completed++;

          // Close connection only after all files are written
          if (completed === categories.length) {
            connection.end();
          }
        });
      });
    });
  });
});
```

Output

```
F:\Kaifclash\fullstack\Node\practical>node lab8.js
Connected to MySQL
Database and table ready.
Movies inserted.
Hollywood.json created
Bollywood.json created
Gujarati.json created
```

9 **Post the data as firstname and lastname using Express and display it in response page.**

Code

Index.js

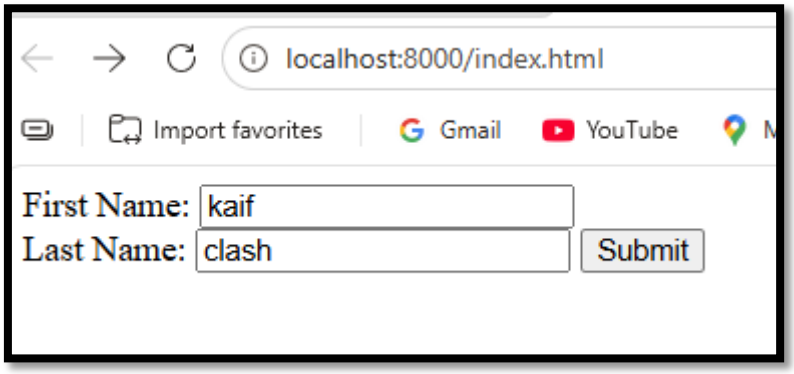
```
var express = require("express");
var app = express();
app.use(express.static("public"));

// bodyParser is require for access posted data
var bodyParser = require("body-parser");
//urlencoded function
var urlencodedParser = bodyParser.urlencoded({ extended: false });

//This will call when user request index.html file.
```

```
app.get("/index.html", function (req, res) {
  res.sendFile(__dirname + "/" + "index.html");
});

//This will call only when user submit data from index.html
app.post("/index.html", urlencodedParser, function (req, res) {
  //   Prepare output in JSON format
  res_data = {
    first_name: req.body.first_name,
    last_name: req.body.last_name,
  };

  res.end(JSON.stringify(res_data));

});
var server = app.listen(8000);
console.log("http://localhost:8000/index.html");
```

Index.html

```
<html>
  <body>
    <form action="http://127.0.0.1:8000/index.html" method="POST">
      First Name: <input type="text" name="first_name" /> <br />
      Last Name: <input type="text" name="last_name" />
      <input type="submit" value="Submit" />
    </form>
  </body>
</html>
```

Output



10	**Read the data from JSON file and display it in HTML template.**

Code

Data.html

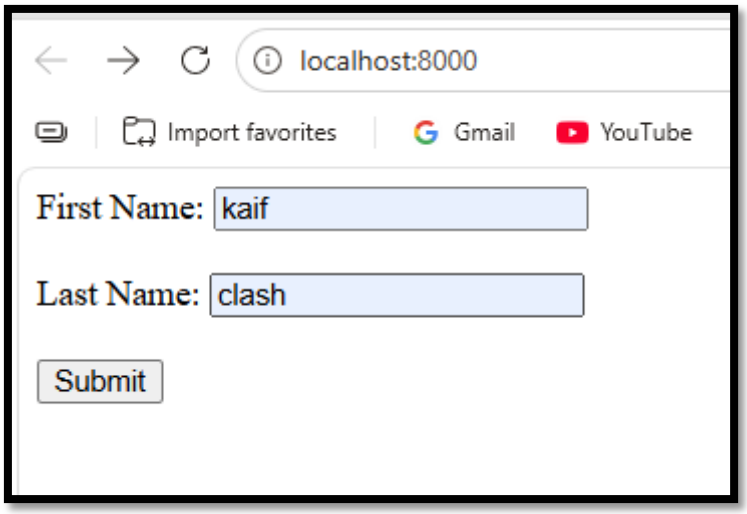
```html
<table align="center" width="50%" border="1">
  <tr>
    <td>First Name</td>
    <td>{{%FNAME%}}</td>
  </tr>
  <tr>
    <td>Last Name</td>
    <td>{{%LNAME%}}</td>
  </tr>
</table>
```

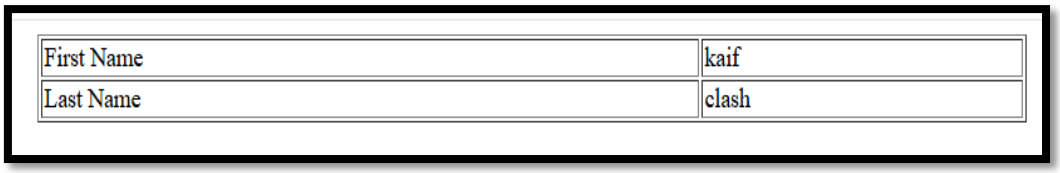
Indexs.html

```javascript
var express = require("express");
var app = express();
var fs = require("fs");

// middleware to read form data
app.use(express.urlencoded({ extended: false }));

// read template once
const htmlData = fs.readFileSync("./data.html", "utf-8");

// show form page (optional)
app.get("/", (req, res) => {
  res.send(`
    <form action="/submit" method="POST">
      First Name: <input type="text" name="first_name"><br><br>
      Last Name: <input type="text" name="last_name"><br><br>
      <button type="submit">Submit</button>
    </form>
  `);
});

// POST route (THIS IS WHERE req EXISTS)
app.post("/submit", (req, res) => {
```

```
  let data = htmlData.replace("{{%FNAME%}}", req.body.first_name);
  data = data.replace("{{%LNAME%}}", req.body.last_name);

  res.send(data);
});

app.listen(8000, () => {
  console.log("http://localhost:8000");
});
```

Output





First Name	kaif
Last Name	clash

Submission Date:24/04/2026 Faculty Signature: